UNIT V · BIG DATA AND SPARK

Apache Spark

A gentle overview of Big Data and Spark: what Spark is, its history, present and future, running it locally, and its basic architecture

SETTING THE SCENE

Big Data in a nutshell

V Volume

Terabytes to petabytes: too big for one machine's disk or memory.

V Velocity

Data arrives continuously: clicks, sensors, transactions, logs.

V Variety

Tables, JSON, text, images, logs: structured and unstructured.

- **Why clusters?** Hardware stopped getting much faster per core around 2005, but data kept growing. The answer: split work across many machines.
- **Why a new engine?** Parallel programming across machines is hard. Engines like MapReduce and Spark hide the hard parts: distribution, scheduling and failures.

THE PROBLEM SPARK SOLVES

Why MapReduce was not enough

MapReduce (and Pig on MapReduce)



Spark



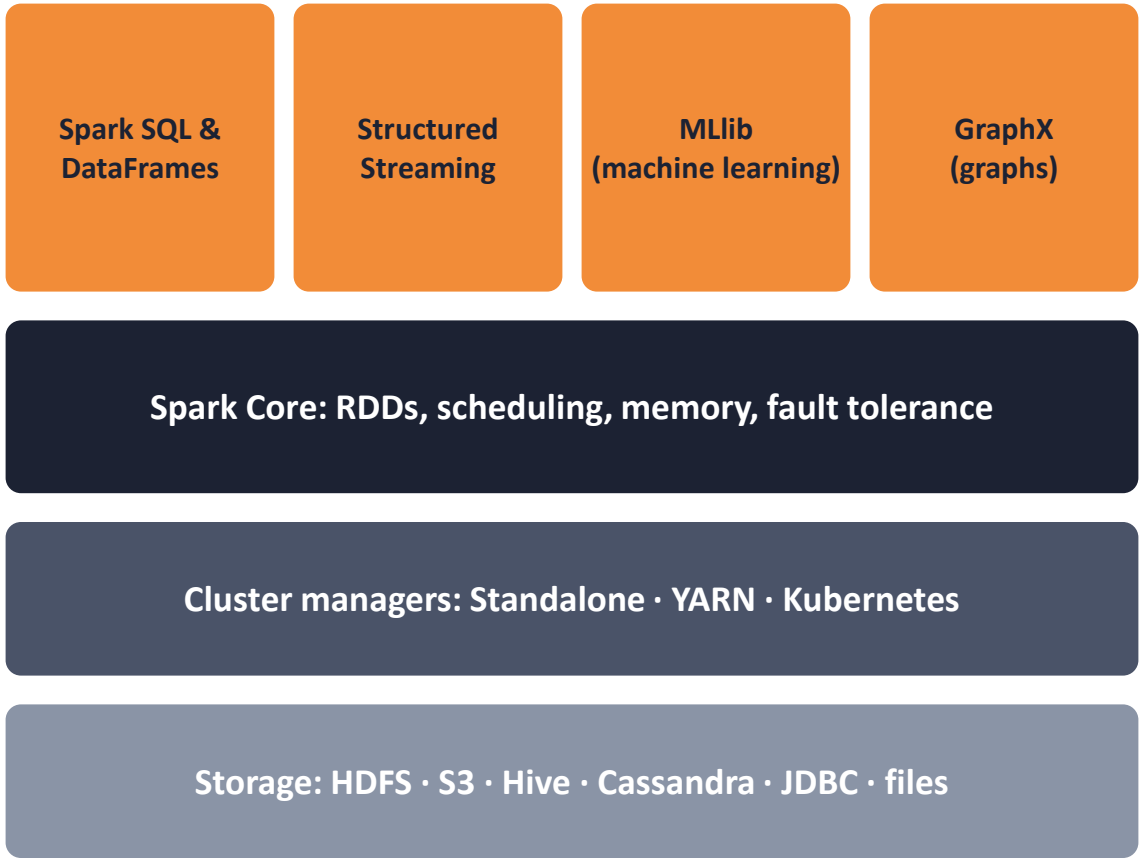
Every MapReduce job writes its output to HDFS before the next job can read it. Iterative work (machine learning, graph algorithms) repeats this many times. Spark keeps intermediate data in memory.

DEFINITION

What is Apache Spark?

A unified computing engine and a set of libraries for parallel data processing on computer clusters.

- **Unified:** one engine for SQL, streaming, machine learning and graphs.
- **Computing engine:** Spark computes; storage lives elsewhere (HDFS, S3, databases).
- **Libraries:** standard libraries plus many third-party packages.



WHY IT WON

Key features of Spark

1 Speed

In-memory processing; often many times faster than MapReduce for iterative work.

2 Ease of use

APIs in Python, Scala, Java, R and SQL. Concise, high-level code.

3 Unified

Batch, SQL, streaming and ML in one program and one engine.

4 Runs anywhere

Laptop, Standalone cluster, Hadoop YARN, Kubernetes, the cloud.

5 Reads anything

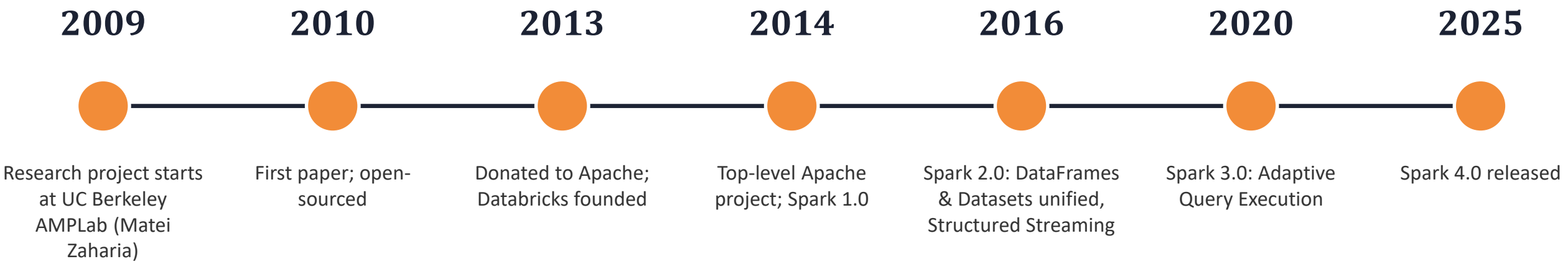
HDFS, S3, Hive, JDBC databases, Kafka, CSV, JSON, Parquet.

6 Fault tolerant

Lost partitions are recomputed from their lineage, not restored from copies.

TIMELINE

History of Spark



Origin story: researchers needed faster iterative machine learning than MapReduce allowed, so they built an engine around in-memory datasets (RDDs).

WHERE SPARK IS TODAY

The present and future of Spark

1 Everywhere in data engineering

Used by thousands of organisations for ETL, analytics and ML; offered by every major cloud.

2 Structured APIs first

DataFrames and Spark SQL are the main way to use Spark; RDDs are the low-level layer.

3 Streaming and ML

Structured Streaming treats a stream as an ever-growing table; MLlib scales ML pipelines.

4 Lakehouse

Spark reads and writes open table formats such as Delta Lake and Apache Iceberg.

5 Spark Connect

Thin clients talk to a remote Spark cluster, making Spark usable from apps and notebooks anywhere.

6 Growing Python focus

PySpark is the most popular API; recent releases keep improving Python support.

GETTING STARTED

Downloading Spark locally

1 Install Java

Spark runs on the JVM. Install a JDK (Java 17 works with current releases).

2 Download

spark.apache.org/downloads : pick a release, package 'Pre-built for Apache Hadoop'.

3 Extract

tar -xzf spark-*.tgz, then set SPARK_HOME and add $SPARK_HOME/bin to PATH.

4 Launch

./bin/pyspark (Python) or ./bin/spark-shell (Scala).

```
$ tar -xzf spark-<version>-bin-hadoop3.tgz
$ cd spark-<version>-bin-hadoop3
$ export SPARK_HOME=$(pwd)
$ export PATH=$SPARK_HOME/bin:$PATH
$ pyspark
```

★ Shortcut: pip or Colab

pip install pyspark gives you Spark for Python without a separate download. Works in Google Colab in about two minutes.

INTERACTIVE AND BATCH

Ways to run Spark

Tool	Language	Use
`pyspark`	Python	Interactive shell; gives you a ready 'spark' object
`spark-shell`	Scala	Interactive shell for Scala
`spark-sql`	SQL	Run SQL queries directly
`spark-submit`	Any	Submit a complete application to a cluster (batch)
`Notebooks`	Python/Scala/SQL	Jupyter, Colab, Databricks and similar

```
$ pyspark --master local[*]

>>> spark
<SparkSession ...>
>>> spark.range(1000)\
...    .toDF("number")\
...    .where("number % 2 = 0")\
...    .count()
500
```

local[*] = run on this machine using all CPU cores. On a cluster you pass --master yarn (or a Kubernetes/Standalone URL).

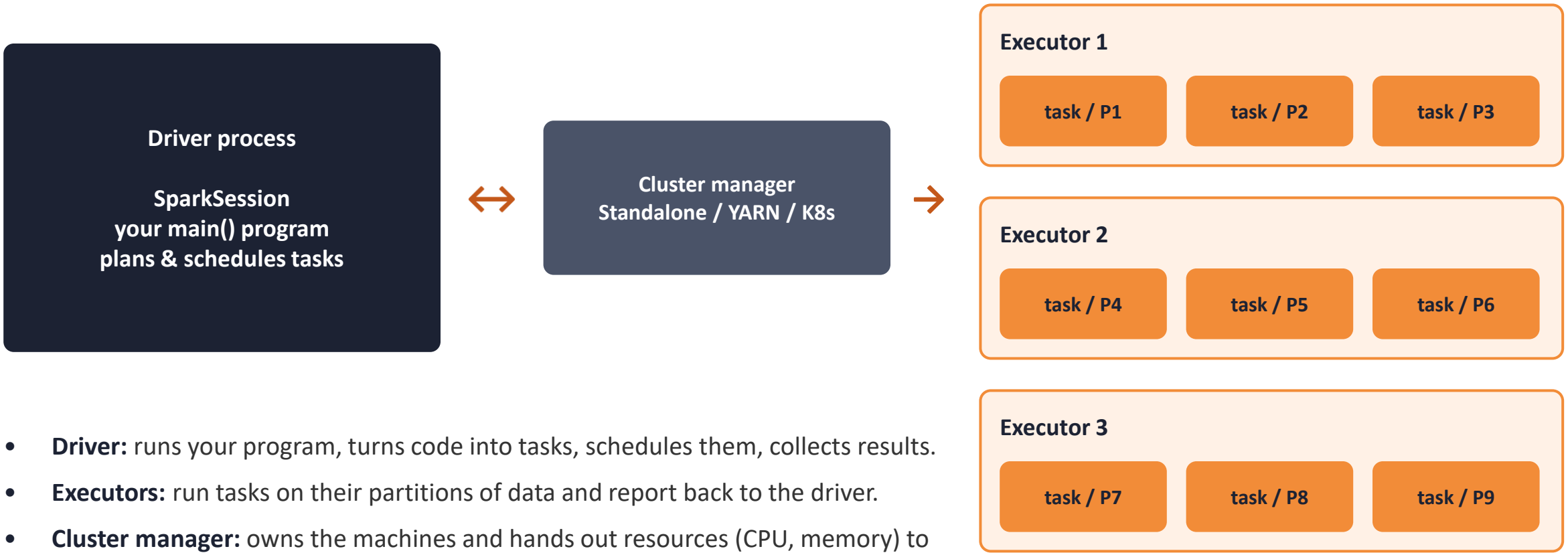


Spark's basic architecture

Driver, executors, cluster manager, and how work gets done

THE BIG PICTURE

Spark's basic architecture



- **Driver:** runs your program, turns code into tasks, schedules them, collects results.
- **Executors:** run tasks on their partitions of data and report back to the driver.
- **Cluster manager:** owns the machines and hands out resources (CPU, memory) to Spark applications.

MAKE IT STICK

The restaurant analogy

1 Head chef = Driver

Reads the order, plans the dishes, splits the work, checks results.

2 Line cooks = Executors

Each cooks their share at the same time and reports back.

3 Restaurant manager = Cluster manager

Decides how many cooks and stoves each chef gets.

4 Ingredient portions = Partitions

Data is split into chunks so every cook works in parallel.

Class simulation: one student is the driver, four groups are executors with a handful of candies each. 'Count the red ones' = parallel tasks, then the driver adds up.

CORE BUILDING BLOCKS

SparkSession, DataFrames and partitions

1 SparkSession

Your entry point. One per application; controls the driver. In pyspark it already exists as spark.

2 DataFrame

A distributed table with named columns and a schema. Like a spreadsheet split across many machines.

3 Partition

A chunk of rows on one machine. Parallelism = how many partitions can be processed at once.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("demo").getOrCreate()

df = spark.range(1000).toDF("number")
print(df.rdd.getNumPartitions())
```

HOW YOU TELL SPARK WHAT TO DO

Transformations and actions

Transformations (lazy)	Actions (trigger work)
filter, where, select	count
withColumn, groupBy, join	show, take, collect
orderBy, distinct, limit	write (save to storage)
Build a plan only	Run the plan and return / save results

N

Narrow transformation

Each input partition feeds one output partition. No data movement. Example: filter, withColumn.

W

Wide transformation (shuffle)

Rows move between partitions across the network. Example: groupBy, join, orderBy.

Wide transformations are the MapReduce shuffle you already know. They are the expensive part of any Spark job.

WHAT HAPPENS WHEN YOU PRESS RUN

Lazy evaluation, the DAG and the Spark UI



- **Lazy:** Spark waits until an action, so it can optimize the whole chain first (like a chef reading the full order ticket before cooking).
- **DAG:** the plan is a directed acyclic graph of stages; stage boundaries are shuffles.
- **explain():** prints the plan without running it. Same idea as EXPLAIN in Pig.
- **Spark UI:** http://localhost:4040 while an application runs: jobs, stages, tasks, storage.



Worked example in Spark

Same data, same question as the Pig deck: Which cities bring in the most revenue?

SPARK STEP 1 · MIRRORS PIG STEP 1

Load and look

```
from pyspark.sql import functions as F

orders = spark.read.csv("orders.csv",
    schema="order_id INT, customer_id INT, "
           "product STRING, category STRING, "
           "qty INT, price DOUBLE")

orders.printSchema()    # like DESCRIBE
orders.show()           # like DUMP
```

1 **New idea**

orders is a DataFrame: a distributed table, the Spark version of a Pig relation.

Reading is lazy until an action such as show() runs.

Same files as the Pig example (no header rows), so a schema is given, just like AS (...) in Pig.

SPARK STEP 2 · MIRRORS PIG STEP 2

Filter rows and compute columns

```
bulk = orders.filter(F.col("qty") >= 2)
bulk.show()

with_amt = (orders
    .withColumn("amount", F.col("qty") * F.col("price"))
    .select("order_id", "customer_id",
            "category", "amount"))
with_amt.show()
```

2 **New idea**

filter = FILTER

withColumn + select = FOREACH ... GENERATE

with_amt feeds every later step, exactly as in Pig.

Both are narrow transformations: each partition is processed on its own, no shuffle.

SPARK STEP 3 · MIRRORS PIG STEP 3

Group, aggregate and see laziness

```
cat_rev = (with_amt.groupBy("category").agg(
    F.sum("amount").alias("revenue"),
    F.count("*").alias("num_orders")))

cat_rev.explain()    # plan only, nothing runs
cat_rev.show()       # action: now it runs
```

category	revenue	num_orders
Electronics	111500.0	4
Furniture	12000.0	2
Stationery	500.0	2

3 **New idea**

No visible 'bag' as in Pig: groupBy must be followed by agg.

groupBy is a wide transformation: it causes a shuffle.

explain() shows the plan; show() executes it.

SPARK STEPS 4 AND 5 · MIRROR PIG STEPS 4 AND 5

Join, sort and take the top N

```
customers = spark.read.csv("customers.csv",
    schema="customer_id INT, name STRING, city STRING")

joined   = with_amt.join(customers, on="customer_id")
city_rev = (joined.groupBy("city")
            .agg(F.sum("amount").alias("revenue")))

top3 = city_rev.orderBy(F.desc("revenue")).limit(3)
top3.show()
```

city	revenue
Chennai	107500.0
Coimbatore	13500.0
Madurai	3000.0

4 **Same answer as Pig**

join(on=...) merges the key into one column, so no :: prefixes.

join, groupBy and orderBy are all wide transformations: three shuffles, but inside one application with no HDFS writes in between.

SPARK STEP 6 · MIRRORS PIG STEP 6

Make it real: spark-submit on YARN

```
# city_report.py
import sys
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("CityReport").getOrCreate()
inp, out = sys.argv[1], sys.argv[2]

orders = spark.read.csv(f"{inp}/orders.csv",
    schema="order_id INT, customer_id INT, product STRING, "
           "category STRING, qty INT, price DOUBLE")
customers = spark.read.csv(f"{inp}/customers.csv",
    schema="customer_id INT, name STRING, city STRING")

top3 = (orders.withColumn("amount", F.col("qty") * F.col("price"))
        .join(customers, on="customer_id")
        .groupBy("city").agg(F.sum("amount").alias("revenue"))
        .orderBy(F.desc("revenue")).limit(3))

top3.write.mode("overwrite").csv(out)
spark.stop()
```

```
$ spark-submit --master yarn \
    city_report.py \
    hdfs:///learn/input \
    hdfs:///learn/output/spark_city

$ hdfs dfs -cat \
  /learn/output/spark_city/part-*
```

6 **vs Pig Step 6**

YARN shows ONE Spark application, not a chain of MapReduce jobs. Intermediate data stays in memory.

The Spark UI (port 4040 while it runs) shows the DAG of stages for this job.

SPARK STEP 7

Beyond Pig: Spark SQL and caching

```
with_amt.createOrReplaceTempView("orders")
customers.createOrReplaceTempView("customers")

spark.sql("""
  SELECT c.city, SUM(o.amount) AS revenue
  FROM orders o JOIN customers c
    ON o.customer_id = c.customer_id
  GROUP BY c.city
  ORDER BY revenue DESC LIMIT 3
""").show()
```

```
joined.cache()

# 1st report: computes and caches
joined.groupBy("city")\
      .agg(F.sum("amount")).show()

# 2nd report: reuses memory
joined.groupBy("category")\
      .agg(F.sum("amount")).show()
```

Same engine, SQL or DataFrame code, identical plan. cache() keeps reused data in memory; Pig would recompute and write to disk again.

PUTTING IT TOGETHER

Pig vs Spark: the connection map

Idea	Pig	Spark (PySpark)
Dataset	`relation (bag of tuples)`	`DataFrame`
Load	`LOAD ... USING PigStorage`	`spark.read.csv(..., schema=...)`
Inspect	`DESCRIBE, DUMP`	`printSchema(), show()`
Pick rows / compute	`FILTER, FOREACH ... GENERATE`	`filter(), withColumn(), select()`
Group / join	`GROUP (bags), JOIN (:: names)`	`groupBy().agg(), join(on=...)`
Sort / top N	`ORDER, LIMIT`	`orderBy(), limit()`
Save / plan	`STORE, EXPLAIN`	`write.csv(), explain()`
Run on cluster	`pig -x mapreduce`	`spark-submit --master yarn`
Under the hood	`Chain of MapReduce jobs, disk between`	`One DAG, in-memory stages`

4

Spark's four libraries

Spark SQL, Structured Streaming, MLlib and GraphX: one engine, four kinds of work

WHERE EACH ONE FITS

One engine, four libraries

Library	What it does	Data it works on
Spark SQL	Query and transform structured data with DataFrames or SQL	Tables (rows and columns)
Structured Streaming	The same DataFrame/SQL operations, on data that keeps arriving	A growing stream of rows
MLlib	Train and apply machine learning models	Feature vectors (numeric columns)
GraphX	Analyze networks: nodes and the connections between them	Vertices and edges

Spark SQL is the library behind every example in this deck so far: filter, withColumn, groupBy, join, and spark.sql(...) are all Spark SQL running on DataFrames.

LIBRARY 2 OF 4

Structured Streaming: watch a folder, not a file

```
from pyspark.sql.types import (StructType, StructField,
    IntegerType, StringType, DoubleType)

schema = StructType([
    StructField("order_id", IntegerType()),
    StructField("category", StringType()),
    StructField("qty", IntegerType()),
    StructField("price", DoubleType())])   # ... etc

stream_df = (spark.readStream.schema(schema)
    .csv("hdfs:///learn/streaming_input"))

revenue = (stream_df
    .withColumn("amount", F.col("qty") * F.col("price"))
    .groupBy("category")
    .agg(F.sum("amount").alias("revenue")))

query = (revenue.writeStream
    .outputMode("complete").format("console")
    .option("checkpointLocation", "~/learn/checkpoint")
    .start())
query.awaitTermination()
```

S

New idea

readStream replaces read; writeStream replaces write. Same groupBy/agg code as Spark SQL, but Spark reruns it on every new batch of files.

!

outputMode("complete")

Reprints the full running total each time, not just the new rows. A checkpointLocation is required so Spark can track progress.

STRUCTURED STREAMING EXECUTION

Running it: step by step

1

1. Create the watch folder

```
hdfs dfs -mkdir -p
/learn/streaming_input
```

2

2. Start the job (terminal 1)

```
spark-submit --master
local[*] streaming_demo.py
```

3

3. It waits, printing an empty Batch 0

```
(leave this terminal open)
```

4

4. Drop a new file (terminal 2)

```
hdfs dfs -put new_orders_1.csv
/learn/streaming_input/
```

5

5. Terminal 1 updates automatically

```
Batch 1 prints new
category totals
```

No restart needed: drop another file with different data and Batch 2 appears with updated totals. Stop with Ctrl+C when done.

LIBRARY 3 OF 4

MLlib: machine learning at scale

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans

orders = spark.read.csv("hdfs:///learn/input/orders.csv",
    schema="order_id INT, customer_id INT, product STRING, "
           "category STRING, qty INT, price DOUBLE")

# MLlib needs one "features" column holding a vector
assembler = VectorAssembler(
    inputCols=["qty", "price"], outputCol="features")
features_df = assembler.transform(orders)

kmeans = KMeans(k=2, seed=1,
    featuresCol="features", predictionCol="cluster")
model = kmeans.fit(features_df)

result = model.transform(features_df)
result.select("order_id", "product", "qty",
              "price", "cluster").show()
```

M

New idea

VectorAssembler packs numeric columns into one features vector. Every MLlib algorithm then follows the same fit() / transform() pattern.

K

What it does here

KMeans groups orders into 2 clusters by qty and price, cheap/small orders in one, the expensive laptops in the other.

LIBRARY 4 OF 4

GraphX: analyzing networks

```
import org.apache.spark.graphx._
import org.apache.spark.rdd.RDD

val vertices: RDD[(VertexId, String)] = sc.parallelize(Seq(
  (1L, "Chennai"), (2L, "Coimbatore"), (3L, "Madurai"),
  (4L, "Bengaluru"), (5L, "Mumbai")))

val edges: RDD[Edge[Int]] = sc.parallelize(Seq(
  Edge(1L, 2L, 5), Edge(2L, 3L, 2), Edge(1L, 4L, 7),
  Edge(4L, 5L, 3), Edge(5L, 1L, 4)))

val graph = Graph(vertices, edges)

graph.inDegrees.collect().foreach(println)

val ranks = graph.pageRank(0.0001).vertices
ranks.join(vertices).sortBy(_._2._1, ascending = false)
     .collect().foreach(println)
```

```
$ spark-shell --master local[*] \
    -i graphx_demo.scala
```

G

No Python API

GraphX exists only in Scala/Java. Run it with spark-shell, bundled with Spark, rather than pyspark.

★

What it computes

In-degree per city, then PageRank: higher scores mean a city is reached by more, or more important, trade routes.

RECAP

Summary: Spark in one slide

1 What

Unified engine + libraries for parallel data processing on clusters.

2 Why

In-memory, fast, easy APIs; fixes MapReduce's disk round trips.

3 History

AMPLab 2009 → Apache 2013 → 1.0 2014 → 2.0 2016 → 3.0 2020 → 4.0 2025.

4 Get started

Java + download, or pip install pyspark; then pyspark / spark-submit.

5 Architecture

Driver + executors + cluster manager; data split into partitions.

6 Execution

Lazy transformations build a DAG; actions run it; wide = shuffle.